

MACHINE LEARNING — LAB MANUAL

Lab 6: Logistic Regression & KNN Classification

A 3-Hour Hands-On Lab — BS Computer Science, 5th Semester

Duration: 3 Hours (independent, hands-on)

Prerequisites: Labs 1–4 (Python/tools, cleaning/EDA, Decision Trees, Random Forest/CV/tuning) on Telco Churn; Logistic Regression and KNN theory (sigmoid/log-odds, decision boundary, distance metrics, choosing K) from the ML course

Tools Required: Python 3.x, Pandas, NumPy, scikit-learn, Matplotlib, Google Colab

Includes: Feature scaling, Logistic Regression, KNN with a K-selection experiment, a 4-model comparison against Labs 3–4, coefficient interpretation, and a graded rubric

Learning Objectives

This lab is NOT a re-introduction to Logistic Regression or KNN theory — you already know the sigmoid function, log-odds, distance metrics, and how K controls model complexity. The goal is to practice implementing both classifiers properly, understanding why feature scaling suddenly matters, and placing them alongside the Decision Tree and Random Forest models you already built in Labs 3–4. By the end of this lab you should be able to independently:

- Explain why feature scaling matters for KNN (and helps Logistic Regression) but didn't matter for Decision Trees/Random Forests
- Train and evaluate a Logistic Regression classifier
- Train and evaluate a KNN classifier, and investigate how K affects performance
- Compare four classifiers (Decision Tree, Random Forest, Logistic Regression, KNN) fairly on the same problem
- Interpret Logistic Regression coefficients in terms of odds
- Select and justify a final model using evidence from across the whole course, not just this lab

Module	Focus Area
1	Problem Definition & Dataset Verification (0–15 min)
2	Feature Scaling & Preparation (15–35 min)
3	Logistic Regression (35–65 min)
4	KNN & Choosing K (65–100 min)

Module	Focus Area
5	Model Comparison (4 Models) (100–130 min)
6	Coefficient Interpretation (130–150 min)
7	Final Model Selection & Conclusion (150–180 min)

Dataset & Problem

You continue with the same Telco Customer Churn dataset and problem from Labs 2–4: predicting whether a customer will churn (Yes/No). Today you add two new classifiers — Logistic Regression and K-Nearest Neighbors — and, for the first time in this course, directly compare four different algorithm families on one problem.

Item	Detail
Dataset name	Telco Customer Churn
Official Kaggle link	kaggle.com/datasets/blatchar/telco-customer-churn
Target variable	Churn (Yes/No)

Before You Start

- Open your Lab 2 notebook/output and locate `clean_churn.csv` (or the equivalent cleaned dataframe), and your Lab 3/4 Decision Tree and Random Forest results.
- Create a new Colab notebook: `lab6_<yourname>_churn_logreg_knn.ipynb`, with the student-info block and Markdown headings described in Submission Requirements.
- You will submit only this one notebook — make sure every section asked for is present before you finish.

Part 1 — Problem Definition & Dataset Verification

Re-establish your cleaned dataset and recall what Lab 3/4 already found, so today's new models have something concrete to be compared against.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>pd.read_csv(...)</code>	Load the cleaned dataset saved in Lab 2
<code>df.shape</code> / <code>df.info()</code>	Confirm rows, columns, and dtypes are as expected

Tasks

1. Load `clean_churn.csv` (or re-run your Lab 2 cleaning) and verify its shape and dtypes.
2. Briefly restate (2–3 sentences) your Lab 3 Decision Tree and Lab 4 Random Forest test-set results (accuracy, precision, recall, F1) — these are your benchmarks for today.

Part 2 — Feature Scaling & Preparation

Decision Trees and Random Forests split on raw feature values, so scale never mattered. Logistic Regression and especially KNN behave very differently: KNN measures straight-line distance between points, so a feature like MonthlyCharges (range ~20–120) would completely dominate a feature like tenure (range 0–72) unless you scale them onto comparable ranges first.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>pd.get_dummies(df, columns=[...])</code>	One-hot encode categorical columns (same as Lab 3/4)
<code>from sklearn.preprocessing import StandardScaler</code>	Import a feature-scaling tool
<code>scaler.fit_transform(X_train) / scaler.transform(X_test)</code>	Fit scaling on training data only, then apply to both sets

Tasks

3. Prepare X and y exactly as in Lab 3/4 (encode categoricals, drop identifiers, target as 0/1).
4. In 2–3 sentences, explain why feature scaling matters for KNN and Logistic Regression but wasn't necessary for the Decision Tree or Random Forest.
5. Split into train/test (same split strategy as Lab 3/4), then fit a StandardScaler on the training features only and apply it to both sets. Why is it important to fit the scaler only on the training data?

Part 3 — Logistic Regression

Train a Logistic Regression classifier on the scaled features and evaluate it exactly as you evaluated the Decision Tree and Random Forest in Labs 3–4.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>from sklearn.linear_model import LogisticRegression</code>	Import the classifier
<code>LogisticRegression().fit(X_train_scaled, y_train)</code>	Trains the model
<code>.predict(X) / .predict_proba(X)</code>	Class predictions / predicted probabilities
<code>accuracy_score, precision_score, recall_score, f1_score, confusion_matrix</code>	Same evaluation tools as Labs 3–4

Tasks

6. Train a LogisticRegression model on the scaled training data. Generate predictions on the scaled test set.
7. Compute accuracy, precision, recall, and F1-score, and plot the confusion matrix.
8. Look at `predict_proba` for a handful of test customers. In 1–2 sentences, explain what these probabilities represent and how `predict()` turns them into a Yes/No decision.

Part 4 — KNN & Choosing K

K controls KNN's complexity the same way `max_depth` controlled your Decision Tree's complexity in Lab 3 — a very small K can overfit to noise, a very large K can underfit by averaging over too many unrelated neighbors.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>from sklearn.neighbors import KNeighborsClassifier</code>	Import the classifier
<code>KNeighborsClassifier(n_neighbors=k)</code>	Creates a KNN classifier with a given K
<code>for k in [1,3,5,7,9,11,15,21]: ...</code>	Loop to train/evaluate multiple values of K
<code>plt.plot(...)</code>	Plot accuracy (or F1) against K

Tasks

- Train KNN classifiers for several values of K (e.g. 1, 3, 5, 7, 9, 11, 15, 21) on the scaled data. Record test-set accuracy and F1-score for each K in a results table.
- Plot accuracy (or F1) against K. Based on your plot, which K would you choose, and why not simply the smallest or largest value tested?
- Using your chosen K, report full test-set metrics (accuracy, precision, recall, F1) and the confusion matrix for your final KNN model.

Part 5 — Model Comparison (4 Models)

For the first time in this course, place all four classifiers you've built side by side: Decision Tree and Random Forest (Labs 3–4) vs. Logistic Regression and KNN (today).

Concepts / Functions You May Find Useful

Concept / Function	Purpose
<code>pd.DataFrame({...})</code>	Assemble a single comparison table across all four models

Tasks

- Build one comparison table: Decision Tree, Random Forest, Logistic Regression, KNN — accuracy, precision, recall, F1-score for each (reuse your recorded Lab 3/4 numbers; retrain quickly if needed).
- Which model performs best overall? Does the ranking change depending on which metric you prioritize (e.g. precision vs. recall)? Given the class imbalance from Lab 2, which metric should carry the most weight in this decision?

Part 6 — Coefficient Interpretation

Like Linear Regression in Lab 5, Logistic Regression's coefficients are directly readable — but here they describe log-odds, not a direct dollar or unit effect, so the interpretation is a little different.

Concepts / Functions You May Find Useful

Concept / Function	Purpose
model.coef_ / model.intercept_	The trained model's coefficients and intercept
np.exp(coefficient)	Converts a log-odds coefficient into an odds ratio

Tasks

14. Extract the coefficients and build a table: Feature | Coefficient | Odds Ratio ($\exp(\text{coefficient})$) | Interpretation. Interpret at least 4 features (e.g. "customers on this contract type have roughly ___ $\times$ the odds of churning, holding other features constant").
15. Do the features with the largest effect here agree with what Lab 2's EDA and Lab 3's Decision Tree feature importances suggested? Note any disagreements.

Part 7 — Final Model Selection & Conclusion

Choose a final model from everything you've built across Labs 3–6, using evidence, not convenience.

Tasks

16. Select your final model from all four candidates and justify the choice in 2–3 sentences, referencing your Part 5 comparison table and Lab 4's cross-validation lesson about not trusting a single number too much.
17. Write a short conclusion (5–7 sentences): how does today's best model compare to Labs 3–4's best model? What did feature scaling change? What are the remaining limitations of your best model, and what would you try next?

Submission Requirements

Submit exactly ONE Google Colab notebook. At the very top, include a student-info block with: Name, Student ID, Section, GitHub Profile, Kaggle Profile, and Dataset information.

Your Notebook Must Contain These Sections

- 1. Problem Definition & Dataset Verification 2. Feature Scaling & Preparation 3. Logistic Regression
- 4. KNN & Choosing K 5. Model Comparison (4 Models) 6. Coefficient Interpretation 7. Final Model Selection & Conclusion

Every table, plot, and metric must be accompanied by a short written interpretation — code with no discussion will not be accepted.

Grading Rubric (20 Marks)

Criteria	Description
Feature Scaling & Preparation	3 — Correct scaling (fit on train only); clear explanation of why it matters here
Logistic Regression	3 — Model trained correctly; all four metrics + confusion matrix; predict_proba understood
KNN & Choosing K	4 — Multiple K values tested; results table/plot correct; K choice justified, not just smallest/largest
Model Comparison	4 — Complete, fair 4-model table; correct identification of the best model per relevant metric
Coefficient Interpretation	3 — Odds ratios correctly computed and interpreted; connected to earlier labs' findings
Final Selection & Conclusion	3 — Evidence-based final choice; thoughtful conclusion connecting back across Labs 3–6

Penalties

- Skipping feature scaling for KNN, or scaling the full dataset before splitting (data leakage): marks deducted in Part 2.
- Evaluating with accuracy only (no precision/recall/F1/confusion matrix) for either new model: marks deducted in Parts 3–4.
- Testing only one value of K, or choosing K with no justification: marks deducted in Part 4.
- Comparison table missing a model or a metric: marks deducted in Part 5.
- Coefficients reported with no odds-ratio conversion or interpretation: marks deducted in Part 6.
- Introducing SVM, Naive Bayes, or other algorithms not covered in this lab: marks deducted — save them for a later lab.